

Technical Report - ERCU: Embedded Robot Controller Unit

Option A: Embedded Robot Controller Development (ERCU)

iHub Robotics, Embedded Engineer Technical Assessment

Submitted by: Sijin P P

1. Project Overview

The Embedded Robot Controller (ERCU) is a sensor-based embedded control system developed on the STM32F446RE. The controller reads data from three sensors - an HC-SR04 ultrasonic sensor for obstacle detection, an MPU6050 IMU for tilt measurement, and a DHT11 sensor for temperature and humidity monitoring, and uses this information to control a servo motor and buzzer.

The firmware is developed using FreeRTOS to organize the application into independent tasks. Instead of implementing the entire application inside a single `while(1)` loop, the firmware is divided into separate tasks for sensor acquisition, fault detection, actuator control, and UART diagnostics. This makes the application easier to manage and allows each task to run independently.

The controller responds to sensor data using predefined thresholds. Obstacle distance, temperature, and sensor status are continuously monitored to determine whether the system is operating in a HEALTHY, WARNING, or FAULT state. Depending on the system state, the servo continues scanning, stops, or activates the buzzer when required.

Obstacle distance is measured using the HC-SR04's echo pulse, captured through a timer input capture interrupt rather than software polling, so the pulse width is timestamped precisely at the hardware level.

To improve reliability, the controller performs a Power-On Self-Test (POST) during startup to verify that all peripherals are functioning correctly. An external interrupt is also used for the emergency stop button, allowing the system to stop the servo immediately without waiting for the normal task execution cycle.

This project demonstrates the practical use of STM32 HAL drivers, FreeRTOS task scheduling, queue-based inter-task communication, timer input capture, PWM generation, external interrupts, and UART-based debugging.

All sensor readings, actuator states, and system status are printed continuously over UART, allowing the system's behavior to be observed in real time without additional debugging hardware. This also makes it straightforward to verify correct operation during testing and demonstration.

2. System Architecture

The ERCU firmware is designed as a layered system using **FreeRTOS**. The application is divided into four independent tasks that communicate through FreeRTOS queues. Each task is responsible for a specific function, making the firmware modular, easier to maintain, and simpler to debug.

The system starts by collecting data from three sensors: the HC-SR04 ultrasonic sensor, MPU6050 IMU, and DHT11 temperature and humidity sensor. These readings are acquired by the **Sensor Task**, which packages the data into a structure and sends it to the **SensorQueue**.

The **Fault Task** receives the sensor data from the queue and evaluates it against predefined thresholds. Based on the obstacle distance, temperature, tilt angle, and sensor status, the system is classified as HEALTHY, WARNING, or FAULT. This centralized approach ensures that all fault decisions are made in one place.

After classification, the Fault Task sends the system status to two separate queues. The **Actuator Task** receives the status through **FaultQueue** and controls the servo motor and buzzer. At the same time, the **UART Task** receives the same status through **FaultQueue2** and prints sensor readings, actuator status, and fault information to the serial monitor. Using two queues allows both tasks to process the same information independently. Each task is responsible for a specific function and communicates with the others through FreeRTOS queues. This keeps sensor acquisition, decision making, and actuator control independent and makes the firmware easier to maintain.

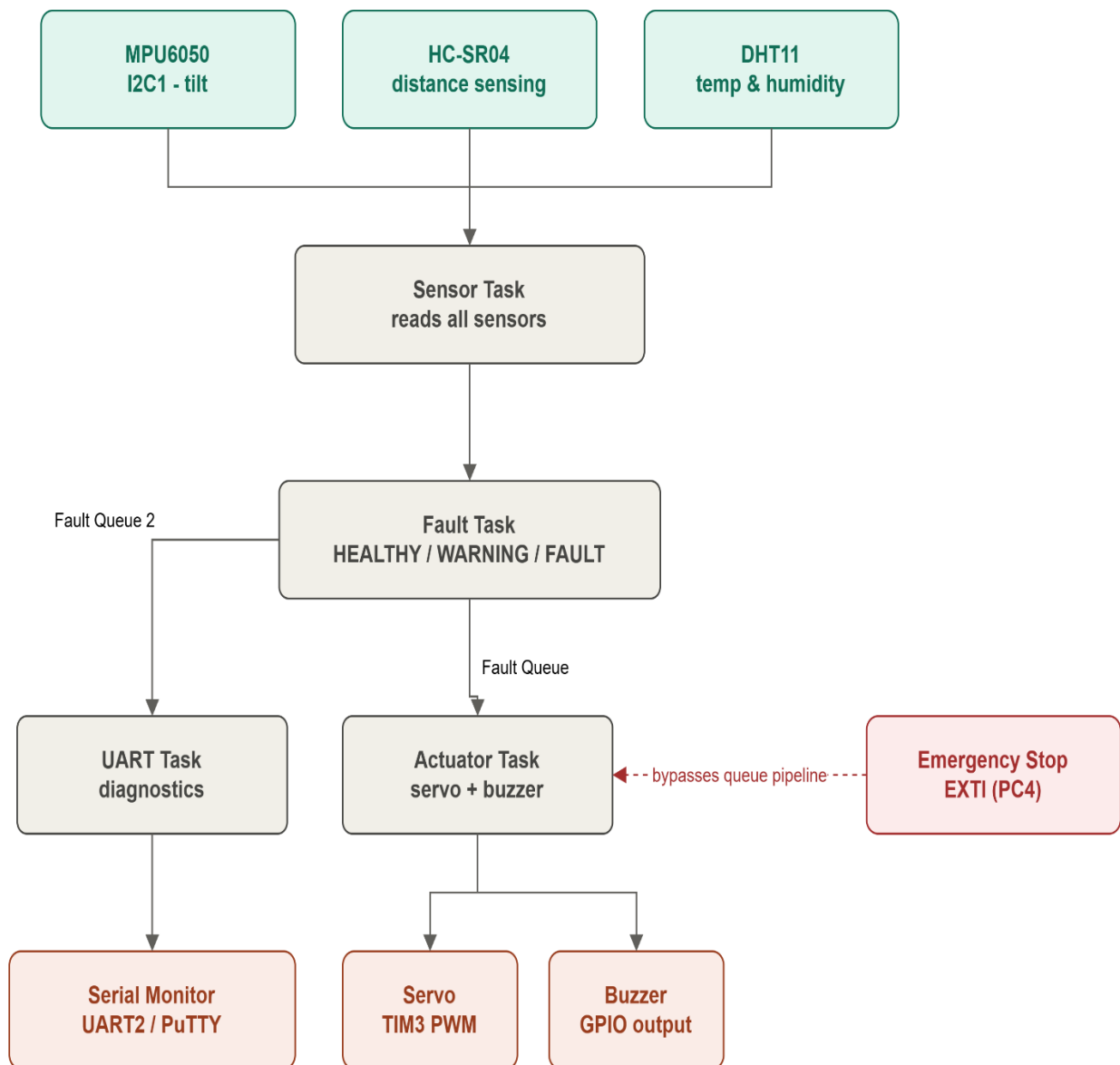
The emergency stop button follows a separate execution path. It is connected to **PC4** as an **EXTI4 external interrupt**. When the button is pressed, the interrupt immediately updates the `emergency_stop_flag`, allowing the Actuator Task to stop the servo without waiting for the normal sensor processing sequence. This provides a faster response during emergency conditions.

2.1 System Data Flow

The system operates in the following sequence:

1. The **Sensor Task** periodically reads data from the HC-SR04, MPU6050, and DHT11 sensors.
2. The collected sensor values are stored in a data structure and sent to the **SensorQueue**.
3. The **Fault Task** receives the sensor data, checks the configured thresholds, and determines whether the system is in a HEALTHY, WARNING, or FAULT state.
4. The classified status is sent to **FaultQueue** for the **Actuator Task** and **FaultQueue2** for the **UART Task**.
5. The **Actuator Task** controls the servo motor and buzzer based on the received status, while the **UART Task** prints the current sensor values and system status to the serial monitor.
6. If the emergency stop button is pressed, the EXTI4 interrupt immediately updates the `emergency_stop_flag`, allowing the Actuator Task to stop the servo without waiting for the queue-based processing path.

The overall architecture of the system is shown below.



2.2 Task Priority

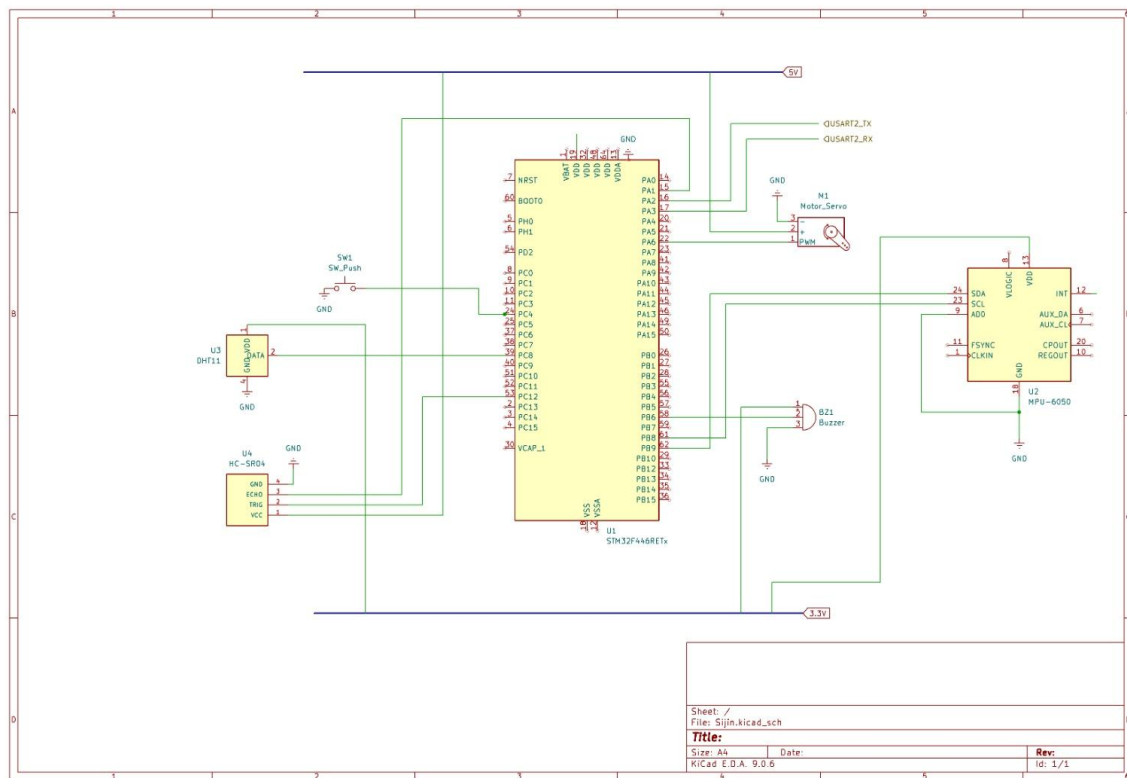
Task	Priority	Responsibility
Sensor Task	High	Reads MPU6050, HC-SR04, DHT11
Fault Task	Above Normal	Evaluates sensor data and determines system state
Actuator Task	Normal	Controls servo and buzzer
UART Task	Low	Prints diagnostics to serial terminal

3. Circuit Diagram

3.1 Hardware Schematic

Figure 3.1 Presents the complete hardware schematic of the ERCU system. It illustrates the connections between the STM32F446RE, sensors, actuators, and the emergency stop button.

Figure 3.1: Complete hardware schematic of the ERCU system



3.2 Hardware Connections

Component	STM32 Pin	Interface	Use
HC-SR04 Trigger	PC12	GPIO Output	Starts ultrasonic measurement
HC-SR04 Echo	PA1	TIM2 Input Capture	Measures echo pulse width
MPU6050 SDA	PB9	I2C1	Transfers IMU data
MPU6050 SCL	PB8	I2C1	I2C clock
DHT11 Data	PC8	GPIO	Temperature and humidity data
Servo Signal	PA6	TIM3 PWM	Controls servo angle
Active Buzzer	PB6	GPIO Output	Fault indication
Push Button	PC4	EXTI4	Emergency stop
UART TX	PA2	USART2	Serial debugging

Component	STM32 Pin	Interface	Use
UART RX	PA3	USART2	Serial debugging

3. Firmware Design

The firmware is divided into separate driver files for each peripheral. Created individual files for the HC-SR04, MPU6050, DHT11, servo motor, buzzer, and push button. This made the project easier to organize because each driver only handles one device, while the FreeRTOS tasks simply call the required functions.

The HC-SR04 and DHT11 both require microsecond-level timing. Since HAL_Delay() only provides millisecond delays, Used the Cortex-M4 DWT cycle counter to generate accurate delays. For measuring the HC-SR04 echo pulse, TIM2 is configured in Input Capture mode. The timer records the rising and falling edges of the echo signal, and the pulse width is then converted into distance.

Before starting the FreeRTOS scheduler, the firmware performs a simple Power-On Self-Test (POST). Each sensor is checked once, and the result is printed over UART. Added this because it helped quickly identifying wiring problems during development instead of spending time debugging the application.

The Sensor Task only collects sensor readings and sends them to the Fault Task. All threshold checking and fault detection are handled inside the Fault Task. This keeps the logic in one place and avoids checking the same condition in different parts of the program. A small hysteresis is also used for obstacle detection so the system does not rapidly switch between WARNING and FAULT when an object is close to the threshold.

4. Testing Methodology

First tested each peripheral separately before integrating everything into FreeRTOS. This made debugging much easier because could confirm that every driver was working before moving to task-based execution.

After integrating the complete system, I tested the project under different operating conditions. During normal operation , verified that the servo motor was sweeping continuously, sensor values were displayed correctly on the serial terminal, and the system remained in the HEALTHY state.

To test the warning conditions , disconnected each sensor one at a time and checked whether the UART displayed the correct warning message. Also increased the temperature value manually during testing to verify the overheat warning.

For fault testing, placed an object in front of the HC-SR04 and gradually moved it closer. The system entered the WARNING state when the object was nearby and switched to the FAULT

state when the distance became less than the configured threshold. Also tested the emergency stop button several times while the servo was moving to make sure it stopped immediately.

5. Results

The system worked as expected during testing. Sensor values were read correctly, and the Fault Task classified the system into HEALTHY, WARNING, or FAULT based on the configured conditions.

When a sensor failed, the system reported a WARNING through UART while continuing normal operation. When an obstacle was detected within the fault threshold, the servo stopped and the buzzer turned on. The emergency stop button also stopped the servo immediately regardless of the current operating state.

The POST routine successfully detected sensor initialization problems during startup, which made hardware verification much easier.

6. Challenges Faced

One issue faced was with UART output. The program compiled successfully, but nothing was printed on the serial terminal. After checking the code, I found that the printf retarget function was not placed correctly. Moving it outside the function solved the problem.

The HC-SR04 driver also caused a problem during startup. The distance value was initially set to zero, so the system entered the FAULT state before the first valid measurement was received. I changed the default value so the system waits for a valid reading before making any decision.

While integrating the FreeRTOS tasks, I also found that both the Fault Task and the Actuator Task were checking obstacle distance independently. This caused inconsistent behaviour because each task could make a different decision. I fixed this by moving all threshold checking into the Fault Task, while the Actuator Task only responds to the received system status.

7. Future Improvements

One improvement would be to use xTaskNotifyFromISR() for the emergency stop instead of checking a flag periodically. This would provide a more direct way of notifying the Actuator Task.

The HC-SR04 readings could also be filtered using a moving average or median filter to reduce measurement fluctuations near the threshold.

The MPU6050 data-ready interrupt can also be used instead of reading the sensor periodically. This would reduce unnecessary I2C communication.

Finally, the UART output could be extended to support CSV or JSON logging so that sensor data can be stored and analyzed on a computer after testing.